MeshPay: HLD & LLD

High-level and low-level design for **upi-offline-mesh**, a Spring Boot backend for offline UPI payments routed through a Bluetooth-style mesh. Repository: github.com/shaswatnaman/upi-offline-mesh

How this was derived. From the README and the source on `main`: the crypto, ingestion, settlement, idempotency, mesh-simulator, demo, key-holder, controller, handler, entity and repository classes, plus `application.properties` and the test class. For the model, controller and repository classes I worked from structured summaries of the files, so exact JSON field names and HTTP status codes for ingest outcomes are marked where I did not confirm them. `WebConfig` and `DashboardController` were not opened. Anything labelled **Proposed** is a design suggestion, not code in the repo.

Contents. Part A, HLD: 1 Scope and actors · 2 Requirements · 3 Architecture · 4 End-to-end flow · 5 Key decisions · 6 Consistency model · 7 Security architecture · 8 Failure modes · 9 Production target. Part B, LLD: 1 Packages · 2 Class diagram · 3 Class catalogue · 4 Data model · 5 Wire formats · 6 API spec · 7 Algorithms · 8 Concurrency · 9 Config and errors · 10 Tests · 11 Patterns · 12 Proposed refactors.

PART A: High-Level Design

What the system is, how the pieces fit, and why it is shaped this way

A1. Scope and actors

MeshPay demonstrates **mesh-routed deferred settlement**: a payment is created and encrypted on an offline phone, carried by strangers' phones, uploaded by whichever phone first reaches the internet, and settled once by the backend.

Actor	Role in the system
Sender (Alice)	Builds a PaymentInstruction, encrypts it with the server's public key, wraps it in a MeshPacket with a TTL.
Relay phones	Untrusted carriers. Hold and forward opaque ciphertext. Can read and rewrite only the outer packet fields.
Bridge phone	Any carrier that gets connectivity. POSTs every packet it holds to the ingest endpoint.
Backend	Owns the keypair, the idempotency gate and the ledger. Decrypts, validates and settles.
Demo operator	Uses the dashboard to inject payments, run gossip, trigger bridge upload and reset.

In scope: ingestion, hybrid crypto, idempotency, replay protection, atomic settlement, an in-process mesh simulator, a demo dashboard. **Out of scope (simulated or absent):** real BLE or Wi-Fi Direct, a real bank or NPCI core, KYC, real PIN verification, sender signatures, bridge authentication.

A2. Requirements

Functional

ID	Requirement
FR1	A sender can produce an encrypted payment packet addressed to the server's public key.
FR2	Packets propagate phone to phone with a bounded hop count (TTL).
FR3	Any internet-connected node can upload the packets it holds through one endpoint.
FR4	Each distinct payment settles exactly once, even if many bridges deliver it concurrently.
FR5	Tampered, stale, future-dated and under-funded packets are rejected with a machine-readable reason.
FR6	Balances, ledger, and mesh state are viewable; the mesh and cache can be reset for repeatable demos.

Non-functional

Quality	Target in this design	How it is met
Confidentiality	Relays cannot read amount or parties	Hybrid RSA-OAEP + AES-256-GCM
Integrity	Any modification is detected	GCM authentication tag
Exactly-once effect	Concurrent duplicates change money once	Atomic claim + DB transaction + optimistic lock
Replay resistance	Old captured packets are useless	signedAt in payload, 24 h window, idempotency cache
Atomicity	Debit, credit, ledger all or nothing	@Transactional
Operability	Runs with only a JDK	H2 in-memory, Maven wrapper
Testability	Race conditions are reproducible	Latch-gated concurrency test

No latency, throughput or availability targets are defined in the repository, so none are claimed here.

A3. Architecture

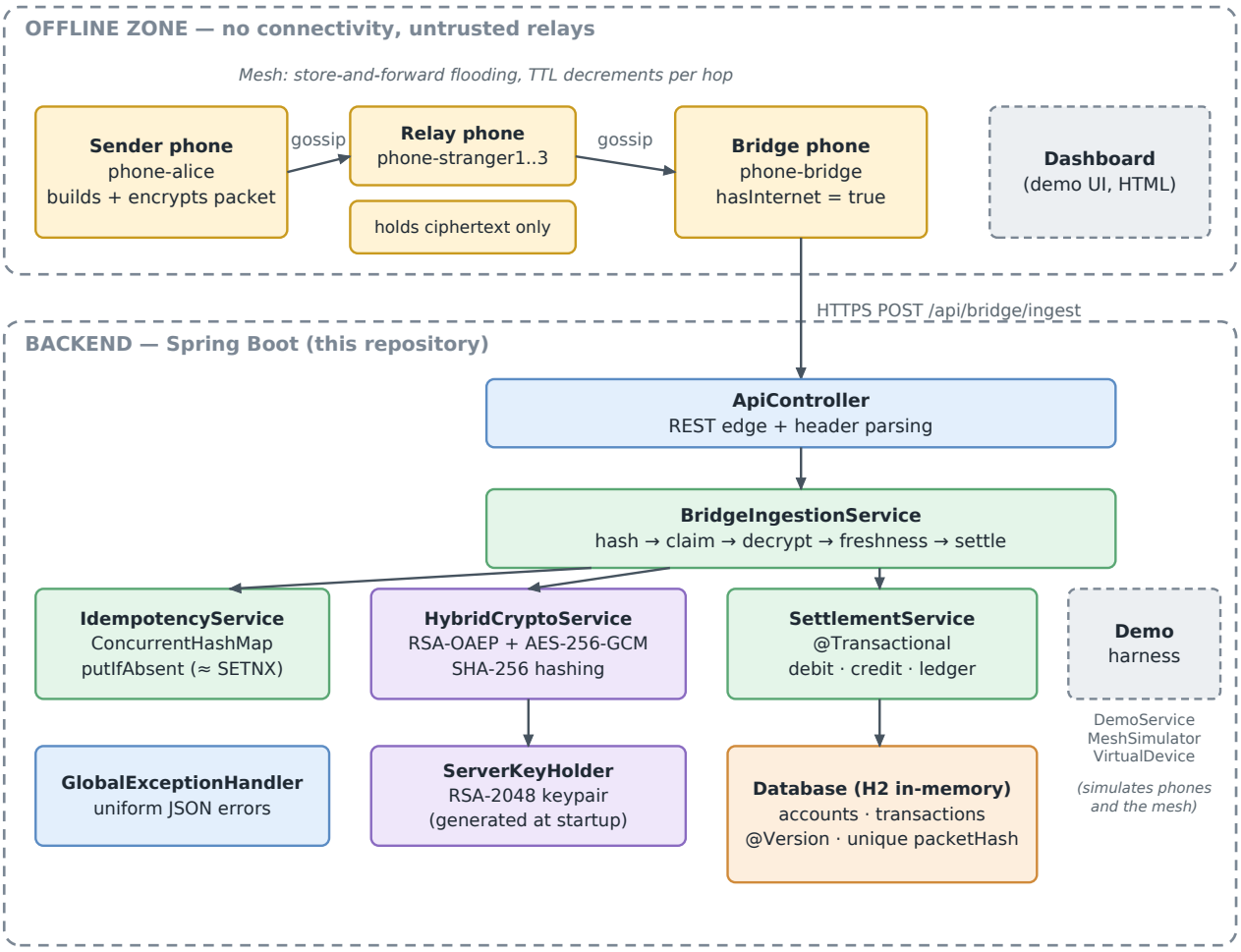


Figure A1. System architecture. The offline zone carries only ciphertext; the backend is the only place plaintext exists.

Component	Responsibility	Tech
ApiController	REST surface: ingest endpoint plus demo, mesh and ledger endpoints. Reads X-Bridge-Node-Id and X-Hop-Count.	Spring MVC
BridgeIngestionService	Orchestrates the pipeline for one packet and maps every outcome to an IngestResult.	Spring service
IdempotencyService	Atomic first-claimer-wins gate on the ciphertext hash, with TTL eviction every 60 s.	ConcurrentHashMap
HybridCryptoService	Encrypt, decrypt and hash. Stateless; creates a Cipher per call.	JCA, Jackson
ServerKeyHolder	Generates the RSA-2048 keypair at startup and exposes it.	java.security
SettlementService	Debits sender, credits receiver and writes a ledger row in one transaction.	Spring Data JPA
MeshSimulatorService + VirtualDevice	Software model of phones, gossip rounds and bridge uploads.	In-process
DemoService	Seeds four accounts and plays the sender phone (builds and encrypts packets).	Spring service

Component	Responsibility	Tech
GlobalExceptionHandler	Turns validation and unexpected errors into uniform JSON.	ControllerAdvice
Database	Accounts and the append-only transaction ledger.	H2 (demo)

A4. End-to-end flow

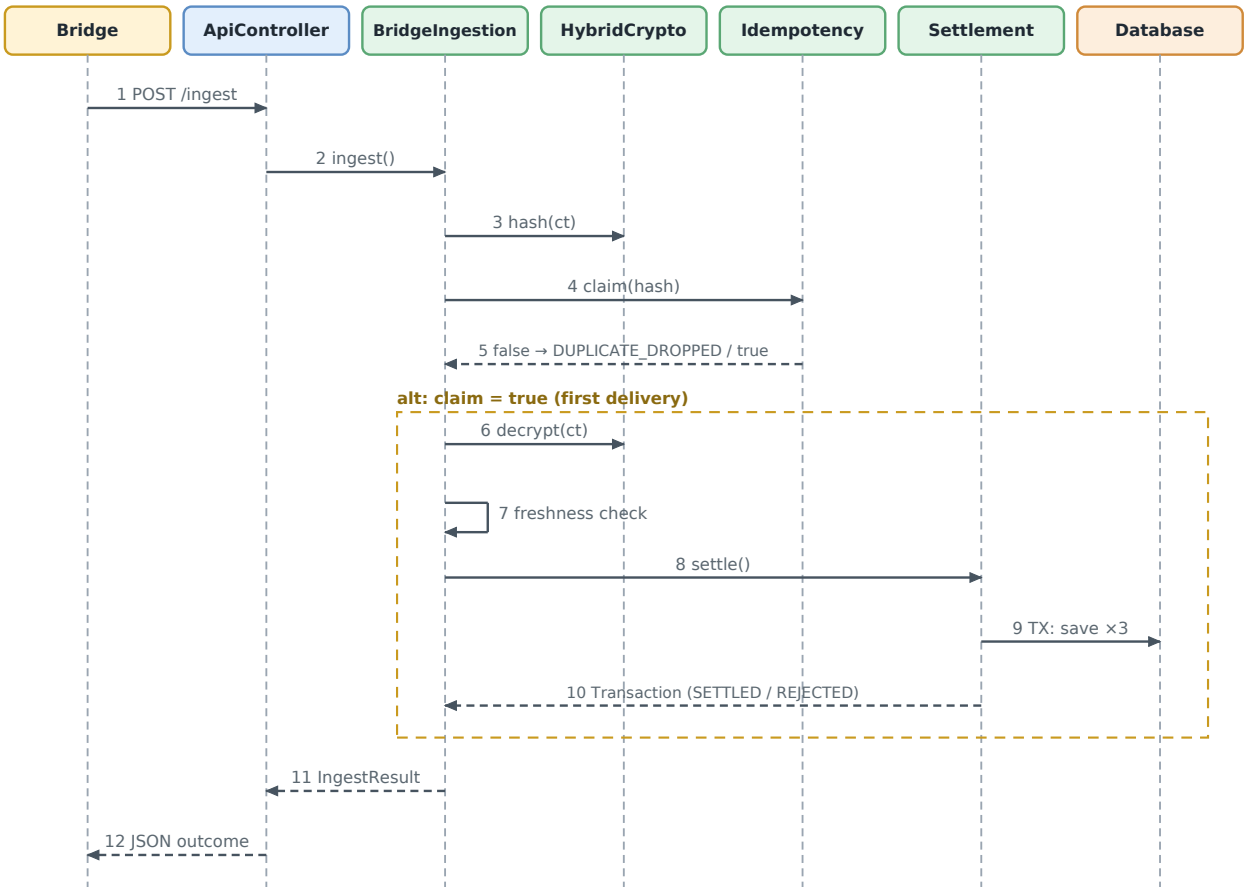


Figure A2. Sequence for one packet through the ingest pipeline. Step 5 short-circuits duplicates before any expensive work.

1. The sender phone creates a PaymentInstruction (VPAs, amount, hashed PIN, random nonce, signedAt) and encrypts it with the server's public key.
2. The MeshPacket (packetId, TTL 5, createdAt, ciphertext) is gossiped; each hop decrements TTL and receivers ignore packetIds they already hold.
3. A device with internet POSTs the packet to `/api/bridge/ingest` with its node id and hop count.
4. The service hashes the ciphertext and calls `claim(hash)`. A false result returns DUPLICATE_DROPPED immediately.
5. The service decrypts. Any failure (wrong key, truncation, tampering) returns INVALID with reason decryption_failed.
6. It checks signedAt: older than 24 h is stale_packet, more than 5 minutes ahead is future_dated.
7. `SettlementService.settle()` runs one transaction: load both accounts, check balance, update both, insert the ledger row.

8 The result (SETTLED, DUPLICATE_DROPPED or INVALID plus reason) is returned as JSON to the bridge.

A5. Key design decisions

Decision	Why	Trade-off or alternative
Hybrid encryption (RSA-OAEP + AES-256-GCM)	RSA alone cannot hold the payload; AES-GCM is fast and authenticated.	Needs a signature layer for sender authenticity (absent).
Idempotency key = SHA-256(ciphertext)	Relays can rewrite packetId but cannot forge ciphertext; copies are byte-identical.	Hash is computed over the base64 string, so re-encoding would change it.
Claim before decrypt and settle	Cheapest rejection of duplicates; avoids repeated RSA decrypts.	A later failure leaves the hash claimed and can lose the payment.
Timestamp inside the encrypted payload	Cannot be altered without breaking the tag.	Depends on phone clocks; needs a skew allowance.
Optimistic locking (@Version)	No row locks; cheap when contention is rare.	Conflicts surface as exceptions and need a retry policy.
One stateless ingest endpoint	Bridges stay dumb; any node can upload anything.	Needs authentication and rate limits in production.
Simulated mesh in the same JVM	Whole flow demoable on one laptop.	Says nothing about real BLE throttling or range.
BigDecimal with scale 2 for money	Exact decimal arithmetic.	Compare with compareTo, not equals.

A6. Consistency model

- **Partition tolerance is forced.** While offline, sender and server cannot talk. The system favours **availability**: the sender can "pay" immediately and the ledger converges later (**eventual consistency, deferred settlement**).
- **A sent payment is an IOU.** Nothing verifies the balance offline. An under-funded packet is stored as REJECTED and reported as insufficient_balance.
- **Double-spend is possible.** A malicious sender can sign two packets against one balance. Whichever reaches the backend first wins. A pre-funded, hardware-attested wallet (UPI Lite style) is the real fix.
- **Delivery semantics.** The mesh gives at-least-once delivery with no ordering. The idempotent consumer converts that into effectively-once settlement.
- **Defense in depth.** Atomic claim, then a unique index on `packetHash`, then an optimistic version check on each account.

A7. Security architecture

Trust boundaries: (1) phone to phone: fully untrusted. (2) bridge to backend: untrusted client over HTTPS, unauthenticated in the demo. (3) inside the backend: trusted.

Threat	Mitigation in the code	Gap
Relay reads the payment	Ciphertext only; AES key wrapped for the server	Packet existence is still metadata

Threat	Mitigation in the code	Gap
Relay edits the payment	GCM tag fails; result is decryption_failed	None for the encrypted part
Relay edits packetId or TTL	Server ignores them; keys on ciphertext hash	TTL can be raised to extend spread
Duplicate or racing delivery	Atomic claim; unique tx index; @Version	Cache is per-JVM
Replay of a captured packet	24 h freshness window plus idempotency cache	Window length is a policy choice
Forged payment from the public key	None: public key is public	**No sender signature**; pinHash is not verified
Junk flood (RSA cost)	Claim happens before decrypt	No rate limit; junk fills the cache
Key compromise or loss	Private key held in memory only	Regenerated each start; not in HSM/KMS
Open admin surfaces	None	H2 console and ingest are unauthenticated

A8. Failure modes

Failure	Effect in the current code	Mitigation direction
Server restarts	Keypair regenerates, so packets encrypted to the old public key can no longer be decrypted. H2 data and the idempotency map are lost.	Persistent key in KMS, key id in packet, persistent DB.
settle() throws after claim	Hash stays claimed; the retry is dropped as a duplicate. Payment lost.	Claim lifecycle (IN_PROGRESS to DONE), release on retryable error.
Optimistic lock conflict	Surfaces as INVALID internal_error; hash stays claimed.	Retry the transaction before giving up.
Unknown VPA or non-positive amount	IllegalArgumentException caught by the outer catch: INVALID internal_error; hash stays claimed.	Map to a distinct, terminal INVALID reason.
Bridge dies mid-upload	Other carriers still hold copies; duplicates are harmless.	None needed; this is the point of the design.
Clock skew	Valid packets may be rejected as stale_packet or future_dated.	Tune the window and skew; use server-side receive time as a cross-check.
Two server instances	Each has its own idempotency map; the DB unique index (per the README) should make the second insert fail and roll back.	Shared claim store (Redis) in front.

A9. Production target (proposed)

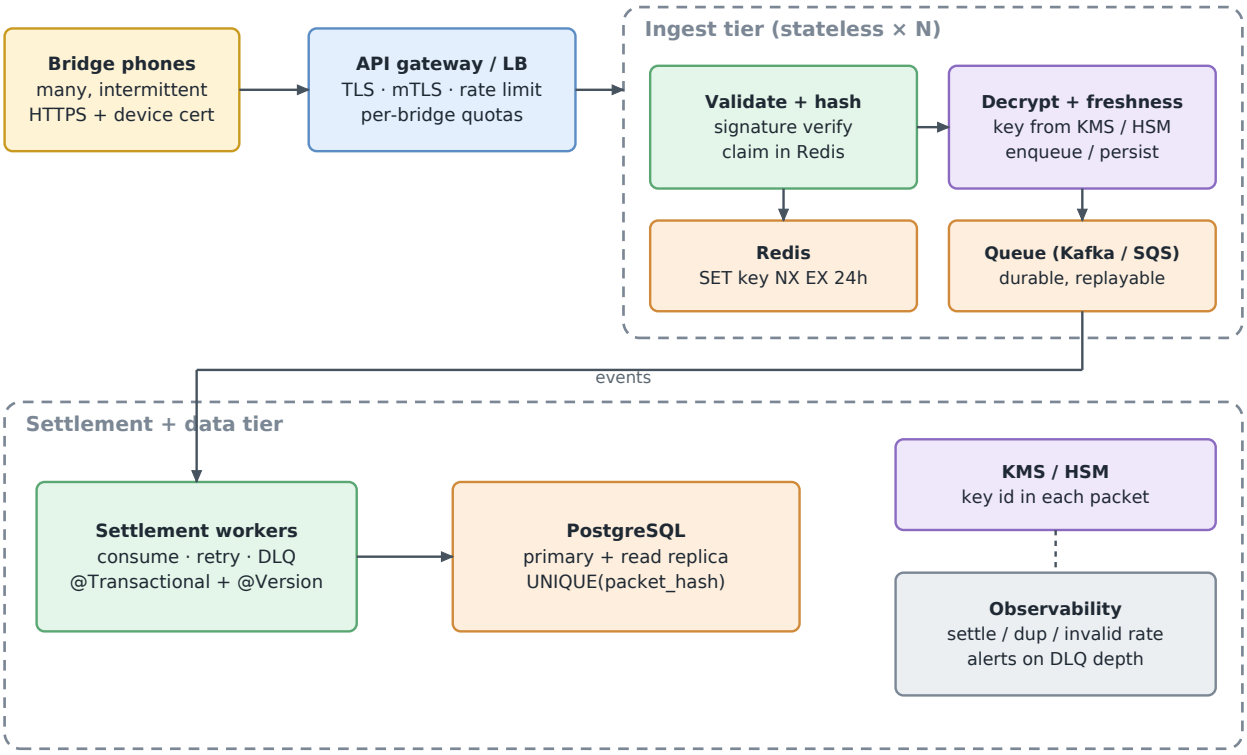


Figure A3. Proposed production architecture. Not present in the repository.

- **Shared idempotency:** Redis `SET key NX EX 86400` for the claim, with a Postgres unique index as the final arbiter.
- **Asynchronous settlement:** ingest verifies, claims and enqueues; workers settle with retries and a dead-letter queue. Return 202 to the bridge.
- **Key management:** private key in KMS or HSM, a key id carried in each packet so keys can rotate.
- **Authentication and abuse control:** mutual TLS or signed bridge certificates, per-bridge rate limits, per-sender velocity checks.
- **Outbox:** publish settlement events from the same transaction that writes the ledger.
- **Metrics:** settled, duplicate and invalid rates; DLQ depth; claim-to-settle latency.

PART B: Low-Level Design

Packages, classes, data model, wire formats, APIs, algorithms and concurrency

B1. Package structure

```
com.shaswatnaman.upimesh
├── UpiMeshApplication            Spring Boot entry point (scheduling enabled for eviction)
├── config/      WebConfig
├── controller/  ApiController, DashboardController
├── crypto/      ServerKeyHolder, HybridCryptoService
├── exception/   GlobalExceptionHandler
├── model/       Account, Transaction, MeshPacket, PaymentInstruction
├── repository/  AccountRepository, TransactionRepository
└── service/     BridgeIngestionService, IdempotencyService, SettlementService,
                 MeshSimulatorService, VirtualDevice, DemoService
resources/  application.properties, templates/dashboard.html
```

Layering is controller → service → repository. The crypto package is a leaf dependency of the services. No service depends on a controller.

B2. Class diagram

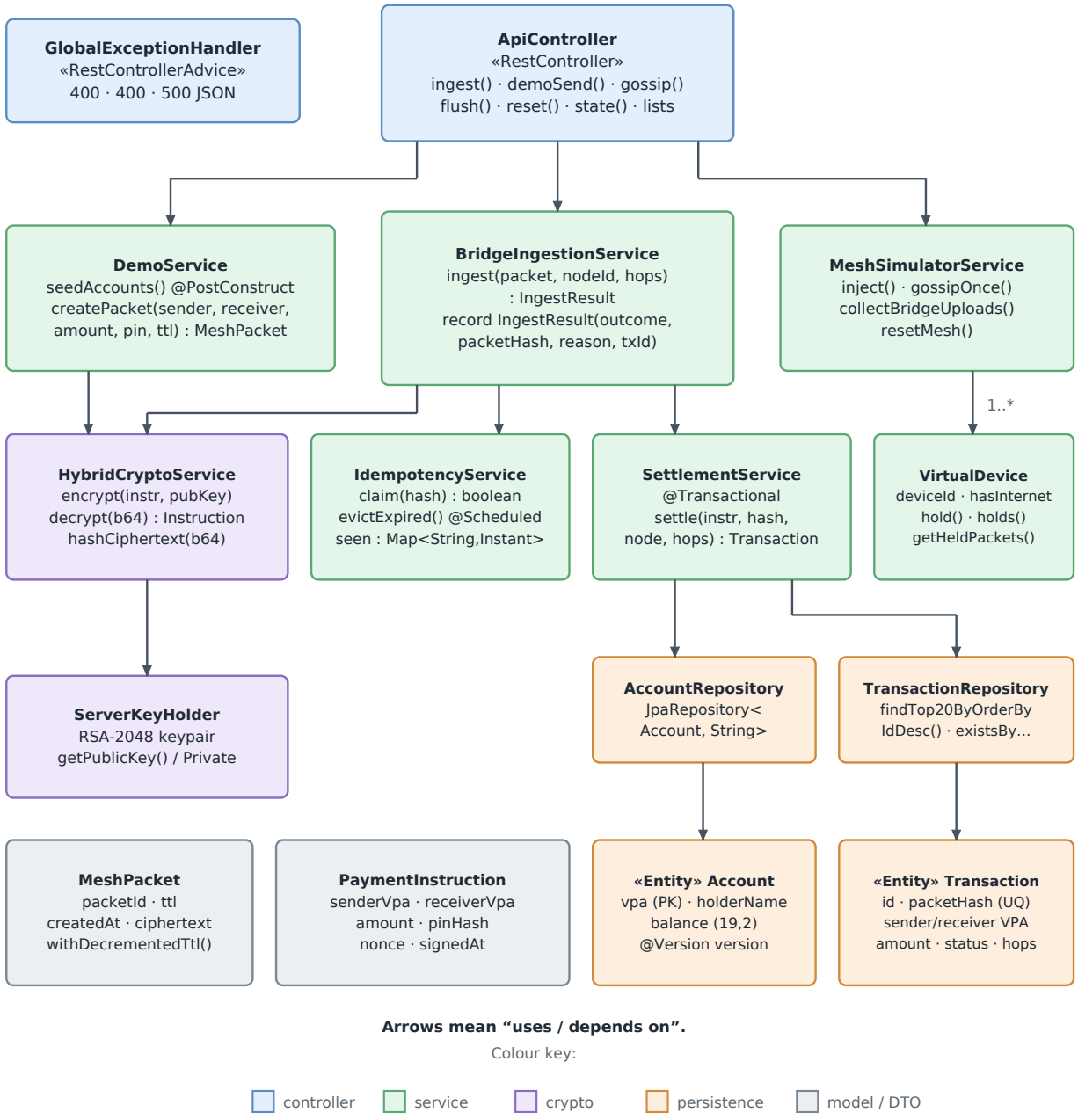


Figure B1. Static structure and dependencies. ApiController also uses IdempotencyService, ServerKeyHolder and both repositories for state and ledger endpoints; DemoService also uses AccountRepository and ServerKeyHolder and builds MeshPacket and PaymentInstruction.

B3. Class catalogue

Services

Class	Responsibility and key members
BridgeIngestionService	**ingest(MeshPacket, String bridgeNodeId, int hopCount) : IngestResult**. Config: `upi.mesh.packet-max-age-seconds` (default 86400) via @Value. Nested `record IngestResult(outcome, packetHash, reason, transactionId)` with factories settled / duplicate / invalid. Catches every Exception and returns INVALID internal_error.
IdempotencyService	**claim(String hash) : boolean** = `seen.putIfAbsent(hash, Instant.now()) == null`. **evictExpired()** every 60 s with TTL from `upi.mesh.idempotency-ttl-seconds`. size() and clear() for state and tests.
SettlementService	**@Transactional settle(PaymentInstruction, packetHash, bridgeNodeId, hopCount) : Transaction**. Loads sender and receiver (IllegalArgumentException if unknown), requires amount > 0, records a REJECTED row if balance is short, otherwise updates both balances and records SETTLED. Private `record(...)` builds the ledger row.
MeshSimulatorService	Holds `Map<String, VirtualDevice>` seeded with phone-alice, phone-stranger1..3 and phone-bridge (hasInternet = true). **inject**, **gossipOnce** (snapshot then flood), **collectBridgeUploads**, **resetMesh**, **snapshotMap**. Records GossipResult and BridgeUpload.
VirtualDevice	One phone: deviceId, hasInternet, a thread-safe map of held packets keyed by packetId. hold() ignores an id it already has; also holds(id), getHeldPackets(), packetCount(), clear().
DemoService	**@PostConstruct seedAccounts()** when the store is empty: alice 5000.00, bob 1000.00, carol 2500.00, dave 500.00 (all @demo). **createPacket(sender, receiver, amount, pin, ttl) : MeshPacket** builds the instruction (SHA-256 of the PIN, UUID nonce, now), encrypts it, wraps it with a new UUID packetId.

Crypto

Class	Responsibility and key members
HybridCryptoService	**encrypt(PaymentInstruction, PublicKey) : String**, **decrypt(String) : PaymentInstruction**, **hashCiphertext(String) : String**. Transformations: `RSA/ECB/OAEPWithSHA-256AndMGF1Padding` and `AES/GCM/NoPadding`; AES-256 key, 12-byte IV, 128-bit tag. decrypt rejects input shorter than 256 + 12 + 16 bytes before any crypto work.
ServerKeyHolder	@Component. @PostConstruct generates the RSA-2048 keypair, logs a 32-character fingerprint of the public key, exposes getPublicKey(), getPrivateKey() and a Base64 helper.

Web

Class	Responsibility and key members
ApiController	Endpoints listed in B6. Delegates all logic; reads the two bridge headers; parallel flush for bridge uploads.
GlobalExceptionHandler	@RestControllerAdvice with three handlers: MethodArgumentNotValidException → 400 validation_error; IllegalArgumentException → 400 bad_request; Exception → 500 internal_error (generic message, logged). Body shape: `{error, message, timestamp}`.

Model and persistence

Class	Responsibility and key members
MeshPacket	Wire object. packetId and ciphertext are @NotBlank; ttl and createdAt are @Min(0). **withDecrementedTtl()** returns a copy with ttl − 1.
PaymentInstruction	Decrypted payload: senderVpa, receiverVpa, amount (BigDecimal), pinHash, nonce, signedAt (epoch ms). pinHash is recorded for realism only; it is not verified.
Account	@Entity, table accounts. vpa is the primary key. balance has precision 19, scale 2. **@Version Long version** enables optimistic locking.
Transaction	@Entity, table transactions, unique index on packetHash. Nested enum Status { SETTLED, REJECTED }. Append-only ledger row.
AccountRepository	JpaRepository<Account, String>.
TransactionRepository	JpaRepository<Transaction, Long> with findTop20ByOrderByIdDesc() and existsByPacketHash(String). The settlement path as read does not call existsByPacketHash.

B4. Data model

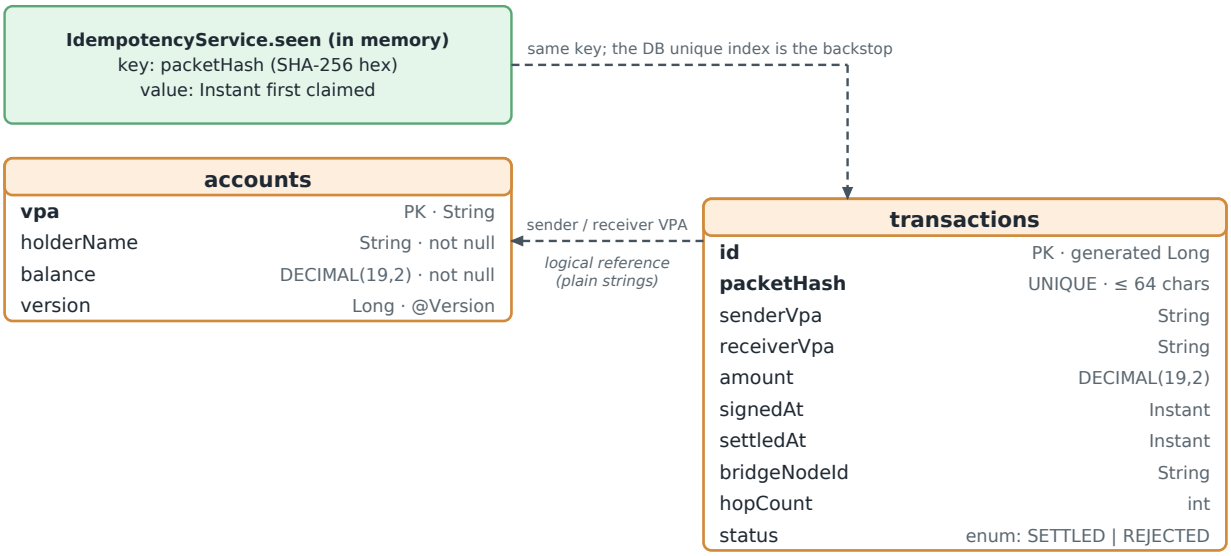


Figure B2. Persistent model (field names shown; physical column names follow Spring's default naming). The in-memory claim map mirrors the ledger's unique key.

Table	Key and constraints	Notes
accounts	PK vpa; balance DECIMAL(19,2) not null; version for optimistic locking	Seeded at startup; ddl-auto = create-drop, so data does not survive a restart.
transactions	PK id (generated); UNIQUE packetHash (≤ 64 chars)	One row per processed payment, SETTLED or REJECTED. hopCount and bridgeNodeId record how it arrived.

B5. Wire formats

MeshPacket (JSON, carried between phones and POSTed to the server)

```
{
  "packetId":   "550e8400-e29b-41d4-a716-446655440000",   // readable and rewritable by relays
  "ttl":        2,                                        // decremented per hop
  "createdAt":  1730000000000,                            // epoch ms, outer field
  "ciphertext": "<base64 of the blob below>"              // opaque, authenticated
}
```

Ciphertext blob (before base64)

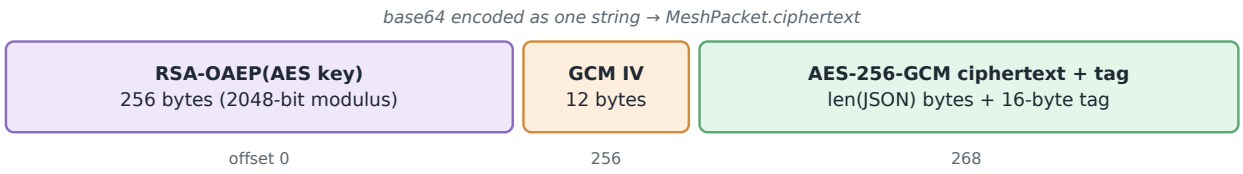


Figure B3. Byte layout. decrypt() slices exactly 256 bytes, then 12, then treats the remainder as ciphertext plus tag.

PaymentInstruction (JSON inside the encryption)

```
{ "senderVpa": "alice@demo", "receiverVpa": "bob@demo", "amount": 500.00,
  "pinHash": "<sha256 hex>", "nonce": "<uuid>", "signedAt": 1730000000000 }
```

The nonce makes two otherwise identical payments encrypt differently. The AES key and IV are random per call, so even identical plaintexts never produce identical ciphertexts; equality of ciphertexts therefore means "the same packet copied", never "re-encrypted".

B6. API specification

Method and path	Request	Response and notes
`POST /api/bridge/ingest`	Headers X-Bridge-Node-Id, X-Hop-Count; body MeshPacket JSON	IngestResult JSON: outcome (SETTLED | DUPLICATE_DROPPED | INVALID), packetHash, reason, transactionId. HTTP status for INVALID not confirmed.
`GET /api/server-key`	none	Server RSA public key, base64.
`POST /api/demo/send`	Sender and receiver VPA, amount, PIN; optional start device (default phone-alice) and TTL (default 5)	Creates and injects a packet into the mesh. Exact JSON field names not confirmed.
`POST /api/mesh/gossip`	none	Runs one gossip round; returns transfer count and per-device packet counts.
`POST /api/mesh/flush`	none	Every bridge node uploads its held packets in parallel to the ingest pipeline.
`POST /api/mesh/reset`	none	Clears all devices and the idempotency cache.
`GET /api/mesh/state`	none	Devices with their packet counts and the idempotency cache size.
`GET /api/accounts`	none	All accounts and balances.
`GET /api/transactions`	none	Latest 20 ledger rows.
`GET /`	none	Dashboard HTML.

Method and path	Request	Response and notes
`GET /h2-console`	none	H2 console (jdbc:h2:mem:upimesh, user sa). Disable in production.

Error body for validation and unexpected failures: `{ "error": "validation_error | bad_request | internal_error", "message": "...", "timestamp": "ISO-8601" }`. validation_error also lists each failing field.

B7. Algorithms

Ingest pipeline

```
ingest(packet, node, hops):
  try:
    h = SHA256(packet.ciphertext)
    if not idempotency.claim(h):          return DUPLICATE_DROPPED
    try:    instr = crypto.decrypt(packet.ciphertext)
    except: return INVALID('decryption_failed')
    age = now - instr.signedAt            # seconds
    if age > maxAge:                      return INVALID('stale_packet')
    if age < -300:                        return INVALID('future_dated')
    tx = settlement.settle(instr, h, node, hops)
    if tx.status == REJECTED:             return INVALID('insufficient_balance')
    return SETTLED(tx.id)
  catch Exception e:                      return INVALID('internal_error: ' + e.message)
```

Settlement (one transaction)

```
settle(instr, h, node, hops):               # @Transactional
  sender   = accounts.findById(instr.senderVpa)   or throw IllegalArgument
  receiver = accounts.findById(instr.receiverVpa) or throw IllegalArgument
  require instr.amount > 0
  if sender.balance < instr.amount:  return record(REJECTED)
  sender.balance   -= amount ; receiver.balance += amount
  accounts.save(sender) ; accounts.save(receiver)     # UPDATE ... WHERE version = ?
  return record(SETTLED)                              # INSERT transactions (unique packetHash)
```

Gossip round

```
gossipOnce():
  snapshot[d] = copy of d.heldPackets for every device d      # freeze the start-of-round state
  for src in devices:
    for pkt in snapshot[src]:
      if pkt.ttl <= 0: continue                               # exhausted copies stop spreading
      for dst in devices:
        if dst == src or dst.holds(pkt.packetId): continue
        dst.hold(pkt.withDecrementedTtl()) ; transfers++
```

Complexity per round is O(devices^2 × packets). Because every device is in range of every other, one round equals many rounds of pairwise gossip, as the source comment notes.

Ingest outcomes

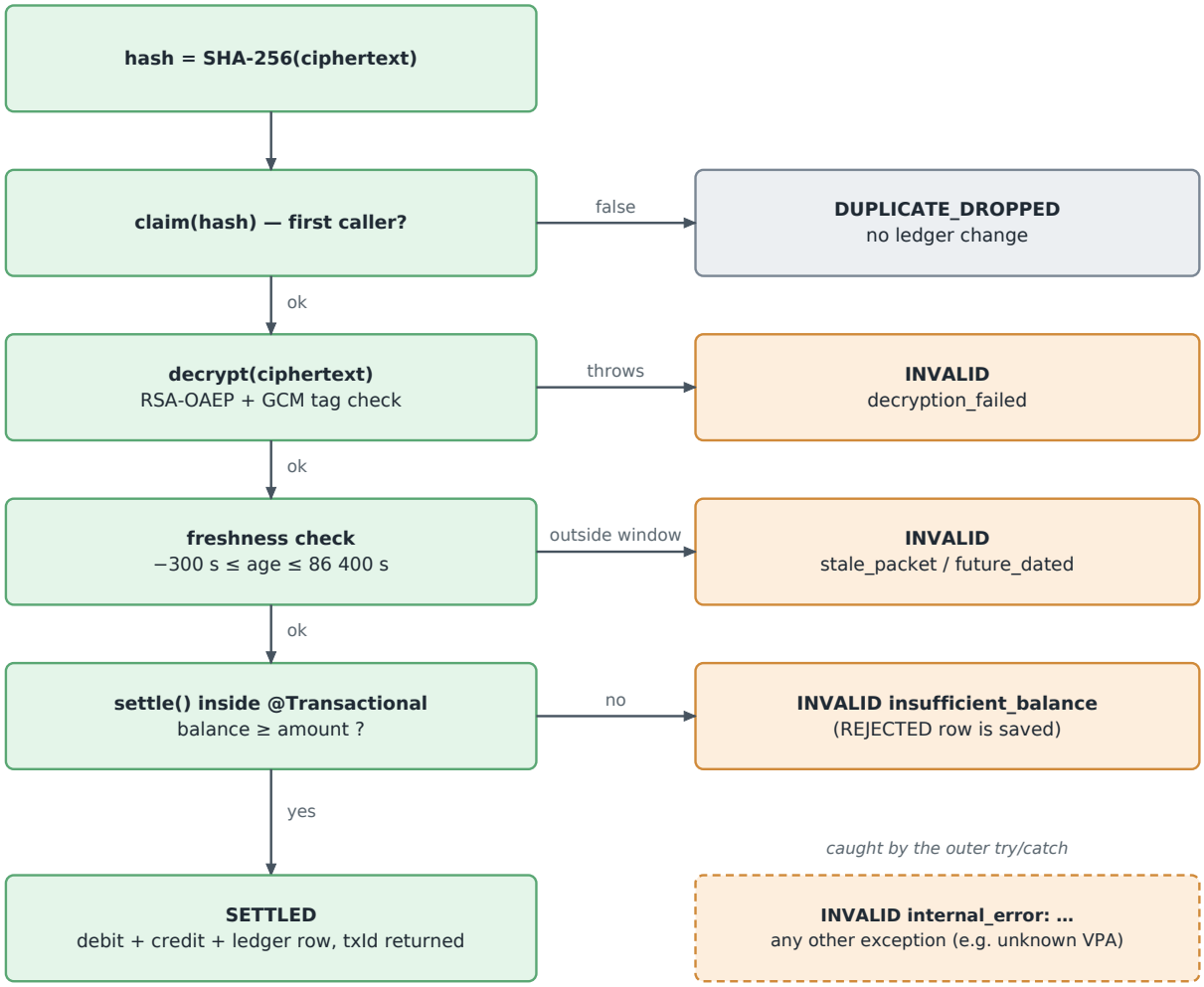


Figure B4. Decision flow and outcomes for a single packet.

Outcome	Reason	Trigger	Ledger effect
SETTLED	none	All checks pass	Balances move; SETTLED row
DUPLICATE_DROPPED	none	claim(hash) returned false	None
INVALID	decryption_failed	decrypt threw (tamper, junk, truncated)	None
INVALID	stale_packet	age > 86 400 s	None
INVALID	future_dated	age < −300 s	None
INVALID	insufficient_balance	balance < amount	REJECTED row only
INVALID	internal_error: …	Any other exception	None (transaction rolled back)

B8. Concurrency design

Component	Shared state	Thread-safety approach
IdempotencyService	ConcurrentHashMap seen	putIfAbsent is atomic; eviction uses removeIf concurrently with claims (weakly consistent iteration is fine here).
HybridCryptoService	SecureRandom, ObjectMapper	Both are thread-safe; a new Cipher is created per call because Cipher instances are not.
ServerKeyHolder	Key pair	Written once in @PostConstruct, read-only afterwards.
SettlementService	Database rows	One transaction per call; @Version makes a conflicting writer fail instead of overwrite.
MeshSimulatorService	Device map	ConcurrentHashMap; gossipOnce works from snapshots. The method itself is not synchronized, so two simultaneous gossip calls may interleave (harmless for the demo).
VirtualDevice	Held packets	Thread-safe map keyed by packetId; hold() ignores duplicates.
Bridge flush	Parallel ingest calls	Safe because the claim is the serialization point for each distinct packet.

B9. Configuration and error handling

Property	Value	Meaning
`server.port`	${PORT:8080}	Env var PORT, default 8080.
`spring.datasource.url`	jdbc:h2:mem:upimesh	In-memory H2, user sa, no password.
`spring.jpa.hibernate.ddl-auto`	create-drop	Schema created at start, dropped at stop.
`spring.jpa.open-in-view`	false	No session held across the view layer.
`spring.h2.console.enabled`	true	Console at /h2-console (demo only).
`upi.mesh.idempotency-ttl-seconds`	86400	How long a claimed hash is remembered.
`upi.mesh.packet-max-age-seconds`	86400	Freshness window for signedAt.

The idempotency TTL is measured from first sight, which is never earlier than signedAt, so a claimed hash always outlives the packet's own freshness window.

B10. Test design

All four tests are in `IdempotencyConcurrencyTest` (`@SpringBootTest`, cache cleared in `@BeforeEach`).

Test	Technique	Assertion
`singlePacketDeliveredByThreeBridgesSettlesExactlyOnce`	3 threads released together by a CountDownLatch, each calling ingest on the same packet	1 SETTLED, 2 DUPLICATE_DROPPED; alice −100.00, bob +100.00 exactly once
`tamperedCiphertextIsRejected`	Flip one character in the middle of the base64 ciphertext	INVALID with decryption_failed

Test	Technique	Assertion
`encryptDecryptRoundTrip`	Encrypt then decrypt a known instruction	Sender, receiver, amount (compareTo) and nonce preserved
`insufficientBalanceIsRejectedNotSettled`	dave (500.00) sends 9999.00	INVALID insufficient_balance; dave's balance unchanged

Not covered: stale and future-dated packets, the optimistic-lock conflict path, settlement failure after a successful claim, unknown VPA, and gossip behaviour.

B11. Design patterns in use

Pattern	Where	Purpose
Pipeline / chain of checks	BridgeIngestionService.ingest	Cheap rejections first, expensive work last.
Idempotent receiver	IdempotencyService	Make at-least-once delivery safe.
Hybrid encryption (envelope)	HybridCryptoService	Asymmetric key transport, symmetric bulk encryption.
Optimistic concurrency	Account.@Version	Detect conflicting writers without locking.
Result object	IngestResult record	Expected failures as values, not exceptions.
Repository	Spring Data JPA interfaces	Persistence behind an interface.
Dependency injection	Constructor injection throughout	Replaceable collaborators; easy to test.
Simulator / fake	MeshSimulatorService, VirtualDevice, DemoService	Exercise the backend without BLE hardware.

B12. Proposed refactors (not in the repo)

1. Claim lifecycle. Replace the boolean claim with a small state machine so a failure does not burn the key, and duplicates can return the original result.

```
interface IdempotencyStore {
    Claim tryClaim(String key);                 // CLAIMED | IN_PROGRESS | DONE(result)
    void  complete(String key, IngestResult r); // terminal: remember the first result
    void  release(String key);                  // retryable failure: allow another attempt
}
// InMemoryIdempotencyStore (ConcurrentHashMap<String, Entry>)  — tests and demo
// RedisIdempotencyStore    (SET key IN_PROGRESS NX EX ttl)      — production
```

```
ABSENT --tryClaim--> IN_PROGRESS --complete--> DONE(result)
                         |
                         +--release (retryable error)--> ABSENT
```

2. Ingest with the lifecycle.

```
claim = store.tryClaim(h)
if claim is DONE:        return claim.result                # replay the original outcome
if claim is IN_PROGRESS: return DUPLICATE_DROPPED           # or 409 / retry-after
try:
    result = process(packet)                                 # decrypt, verify, freshness, settle
    store.complete(h, result)                                # terminal, including INVALID
    return result
except RetryableException:
    store.release(h) ; raise
```

3. Sender authentication step. Add a `SenderSignatureVerifier` between decrypt and freshness: the sender signs the canonical payload (ECDSA or Ed25519); the server verifies against a registered `DeviceRegistry` key for senderVpa. Reject with `invalid_signature`.

4. Settlement hardening. Insert the ledger row first inside the transaction so the unique index is the arbiter; retry on OptimisticLockException a bounded number of times; lock accounts in a fixed order (for example sorted by VPA) if pessimistic locking is ever used; map unknown-VPA and non-positive amounts to a terminal reason.

5. Key management. Introduce a `KeyProvider` interface (local keypair for tests, KMS-backed for production) and a key id in the packet header so decrypt selects the right private key and old packets survive rotation or restart.

Sources

github.com/shaswatnaman/upi-offline-mesh (README)

.../crypto/HybridCryptoService.java, ServerKeyHolder.java

.../service/BridgeIngestionService.java, IdempotencyService.java, SettlementService.java, MeshSimulatorService.java, DemoService.java, VirtualDevice.java

.../controller/ApiController.java, .../exception/GlobalExceptionHandler.java, .../model/*.java, .../repository/TransactionRepository.java

.../resources/application.properties, .../test/.../IdempotencyConcurrencyTest.java